

Object: Programming Exercises - SS2025
Supervised by Mr. Ralf-Oliver Mevius

BOOKSTORE ADMINISTRATION PROJECT

Group Members:

Tuan Viet Nguyen - 1617863

Tieu Cuong Luong - 1616329

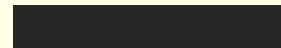
Nguyen Thien Nguyen - 1615629

Hoang Yen Ngoc Nguyen - 1614693



OUTLINE

- A. SYSTEM OVERVIEW & ARCHITECTURE
- B. DATA MODELING & DATABASES
- C. PACKAGE DESIGN & PATTERNS
- D. USER INTERFACE & CONFIGURATION
- E. USER EXPERIENCE DEMONSTRATION



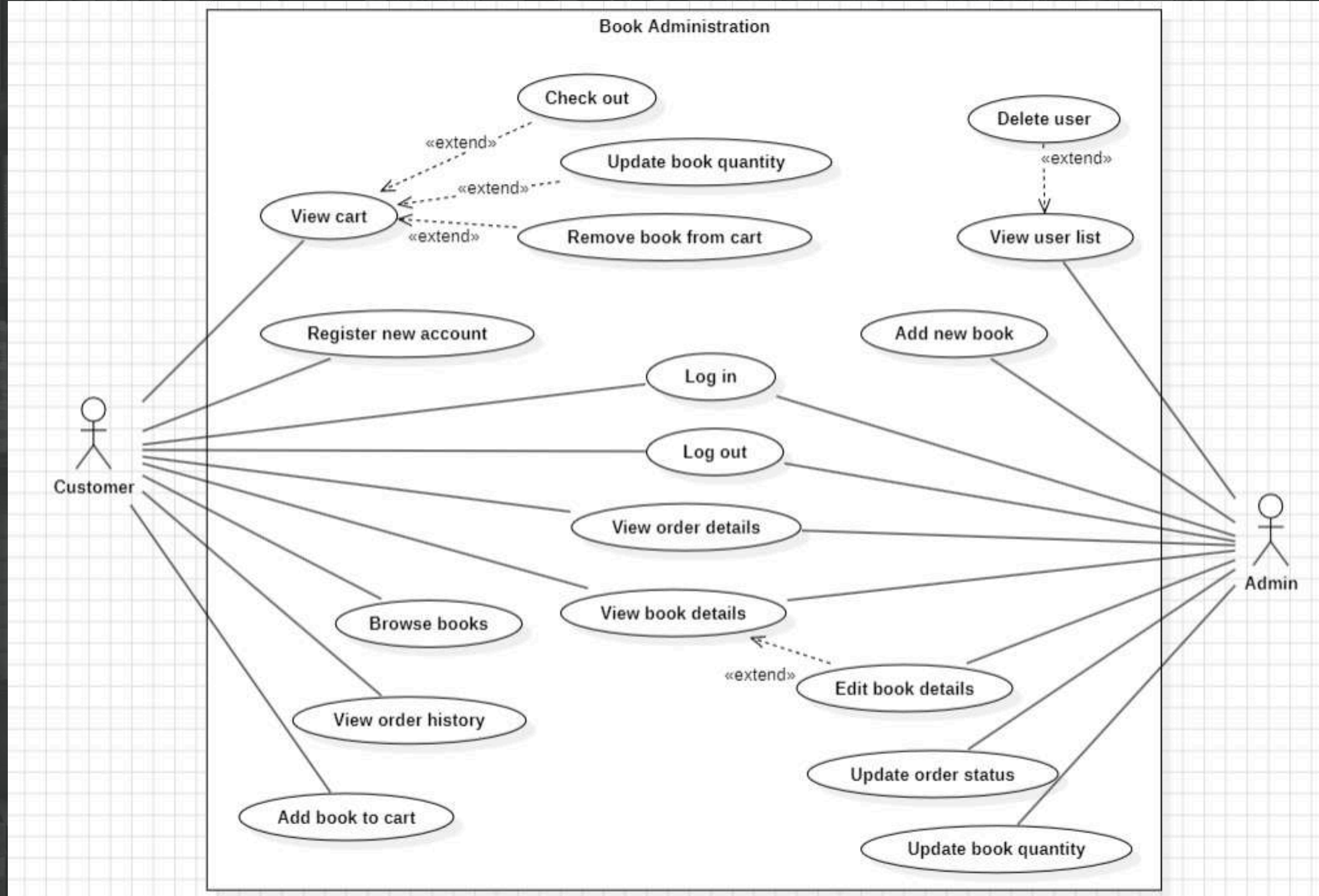
A. SYSTEM OVERVIEW

**PROBLEM: PHYSICAL SPACE, INVENTORY MANAGEMENT,
OPERATION COST.**

SOLUTION: WEB-BASED BOOKSTORE.

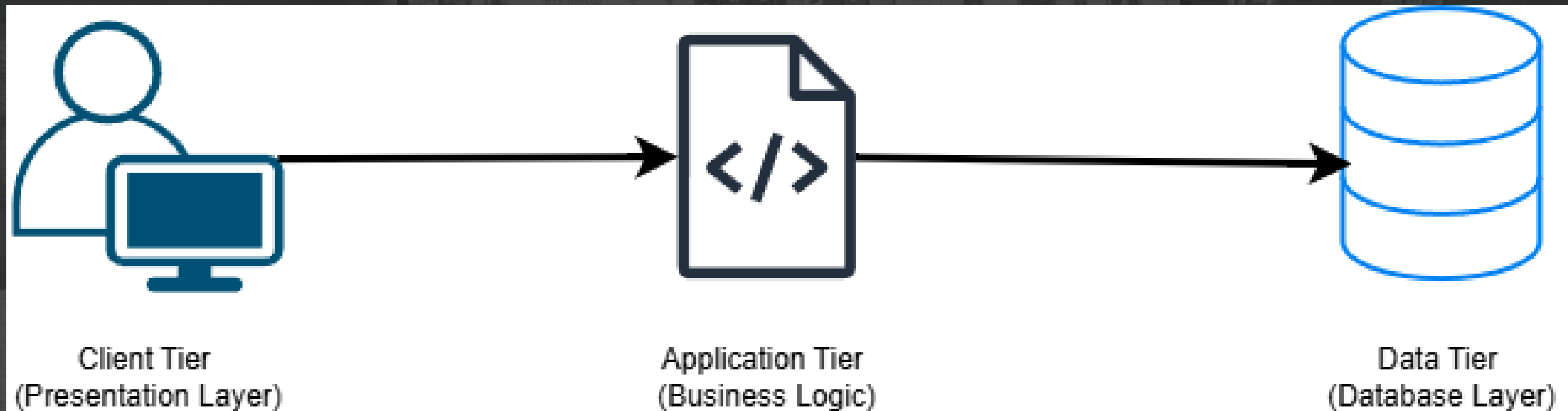
**BENEFITS: STREAMLINED PURCHASING, LABOR-SAVING,
ECO-FRIENDLY SOLUTION.**

USE CASE DIAGRAM



A. SYSTEM ARCHITECTURE

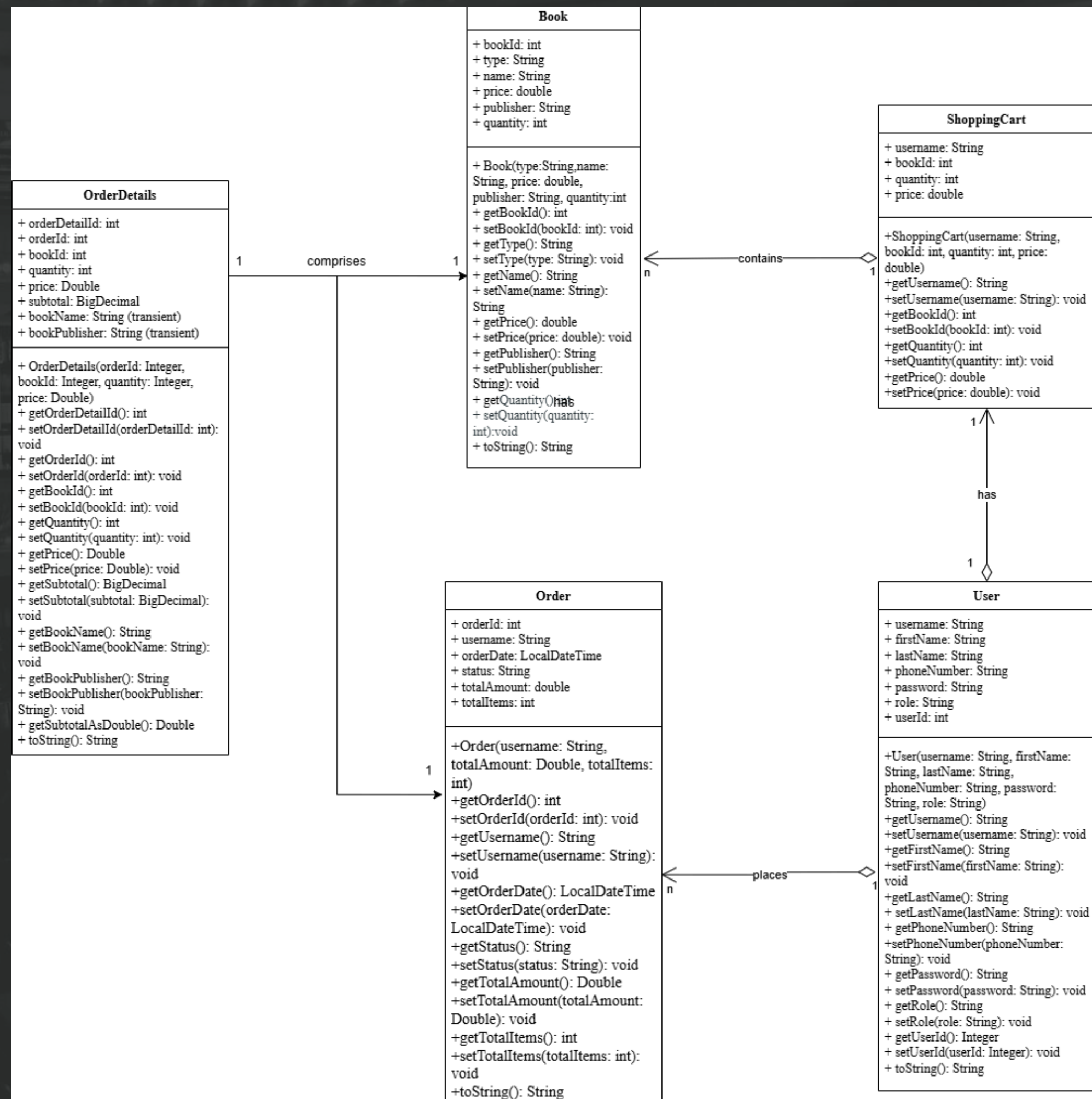
B. 3-TIER CLIENT-SERVER ARCHITECTURE



B. DATA MODELING & DATABASES

1. DATA MODEL

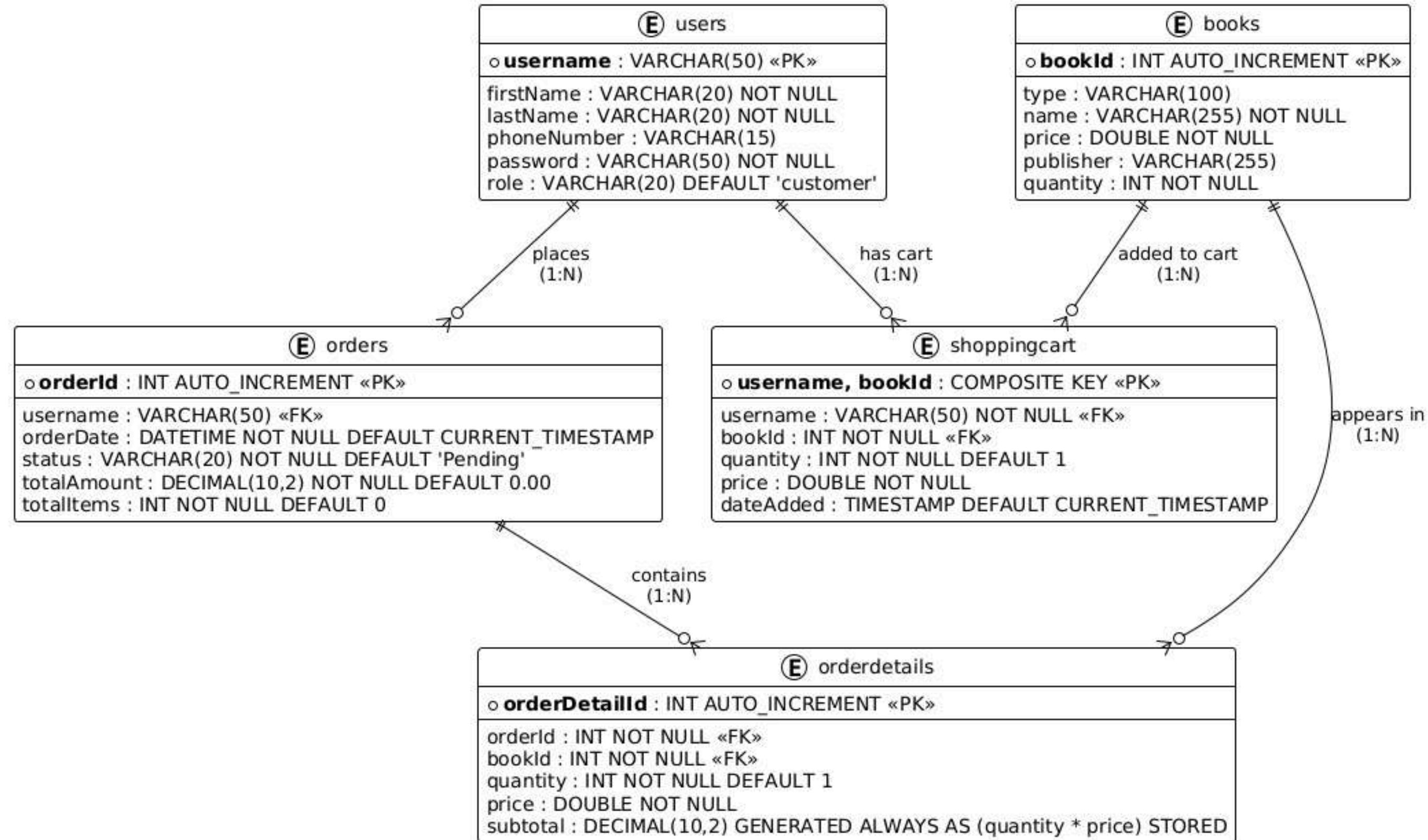
- BOOK
- USER
- SHOPPINGCART
- ORDER
- ORDERDETAILS



B. DATA MODELING & DATABASES

2. DATABASES

Bookstore Management System - Database Schema (ERD)



C. PACKAGE DESIGN & PATTERNS

1. INTRODUCTION - MODULAR ARCHITECTURE OVERVIEW

- **CONTROLLER PACKAGE**
- **DAO PACKAGE**
- **MODEL PACKAGE**
- **USER REPOSITORY**

DAO PACKAGE - DUAL DATABASE APPROACH

```
@Component
public class Database {

    @Value("${spring.datasource.url}")
    private String url;

    @Value("${spring.datasource.username}")
    private String username;

    @Value("${spring.datasource.password}")
    private String password;

    public Connection getConnection() throws SQLException {
        return DriverManager.getConnection(url, username, password);
    }
}
```

```
@Repository
public class BookDAO {
    private List<Book> books;
    private final Database database;

    @Autowired
    private JdbcTemplate jdbcTemplate;
    public BookDAO(Database database) {
        this.database = database;
    }
}
```

MODEL PACKAGE - JPA IMPLEMENTATION

```
@Entity
@Table(name = "books")
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer bookId;

    // Business logic in constructor
    public Book(String type, String name, Double price...) {
        this.type = type;
        this.price = price > 0 ? price : 0; // Validation
    }
}
```

CONTROLLER PACKAGE - RESTFUL API

```
30
31 @RestController
32 @RequestMapping({" /api/user-cart", "/api/cart"}) //
   Support both endpoints
33 @CrossOrigin(origins = "*")
34 public class UserCartController {
35
36     private static final Logger logger = LoggerFactory.
       getLogger(UserCartController.class);
37
38     @Autowired
39     private ShoppingCartDAO shoppingCartDAO;
40
41     @Autowired
42     private BookDAO bookDAO;
43
```

USER REPOSITORY PATTERN - SPRING DATA JPA

```
8
9  @Repository
10 public interface UserRepository extends
11 JpaRepository<User, String> {
12     Optional<User> findByUsername(String
13     username);
14     Optional<User> findByUsernameAndPassword
15     (String username, String password);
16 }
```

```
@Autowired
private UserRepository userRepository;

@PostMapping("/login")
public ResponseEntity<Map<String, Object>> login(@RequestBody
Map<String, String> loginRequest) {
    Map<String, Object> response = new HashMap<>();

    try {
        // Find user by username
        Optional<User> userOpt = userRepository.findByUsername
        (username);

        if (userOpt.isPresent()) {
            User user = userOpt.get();

            // Validate password
            if (user.getPassword().equals(password)) {
                response.put(key:"success", value:true);
                response.put(key:"message", value:"Login
                successful");
            }
        }
    }
}
```

D. USER INTERFACE & CONFIGURATION

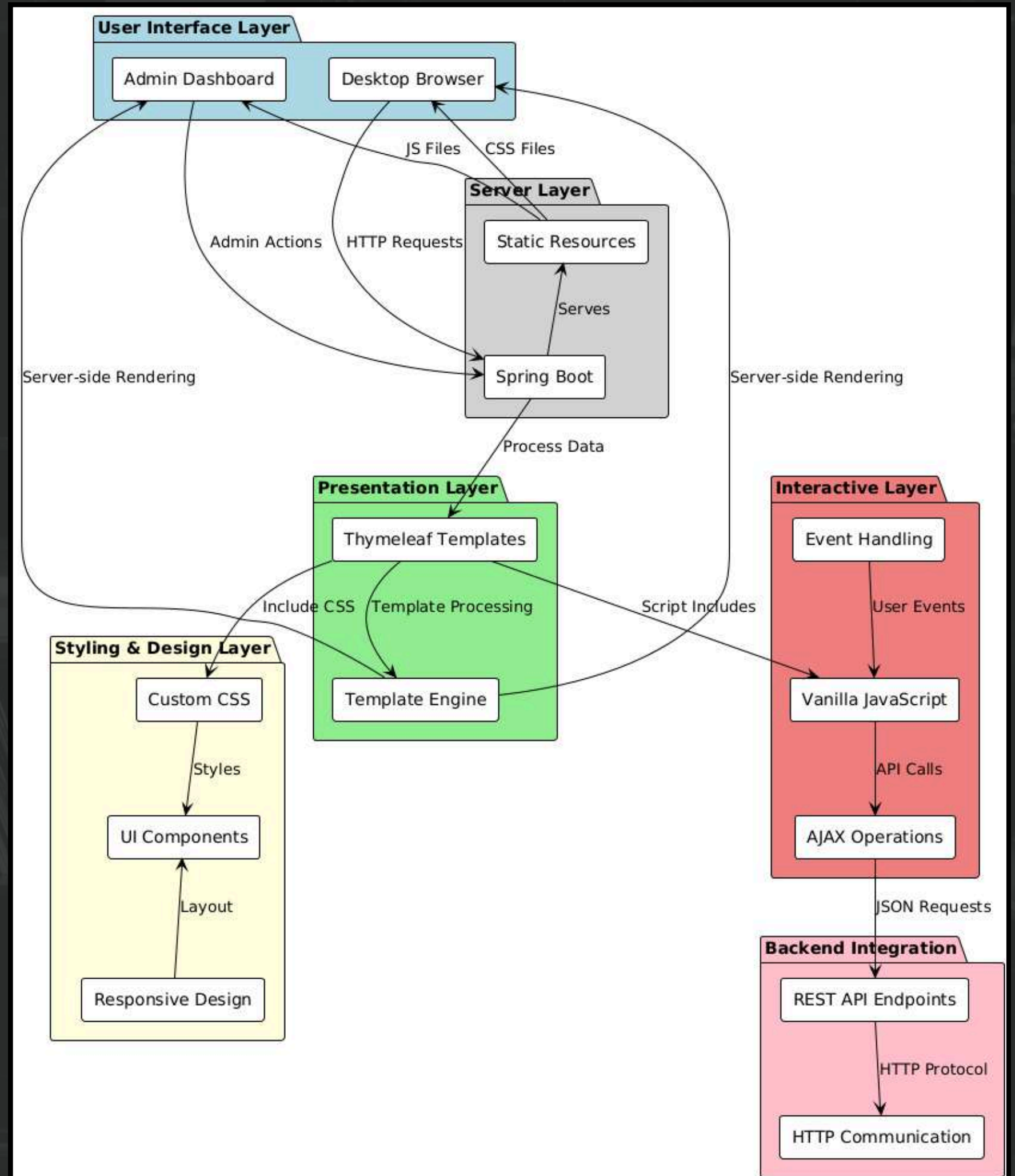
1. FRONTEND DESIGN & ORGANIZATION

Component Type	Customer Interface	Admin Interface
HTML Templates	• index.html (Home) • cart.html (Shopping Cart) • checkout.html (Order Process) • login.html (Authentication) • register.html (Account Creation) • my-orders.html (Order History) • order-details.html (Order View)	• admin.html (Dashboard) • Unified management interface • Tabbed sections for entities
JavaScript Modules	• books.js (Catalog browsing) • cart.js (Cart functionality) • cartManager.js (Cart logic) • checkout.js (Payment process) • auth.js (Authentication)	• admin.js (Dashboard functions) • Order management • User administration • Inventory control
CSS Styling	• styles.css (Global styling) • cart.css (Cart-specific) • Responsive design • Customer-focused UI	• admin.css (Dashboard styling) • Management interface • Data table styling • Admin-specific components
User Flow	Linear Shopping Process: 1. Browse Books 2. Add to Cart 3. Review Cart 4. Checkout 5. Order Confirmation 6. Order Tracking	Management Workflow: 1. Dashboard Overview 2. User Management 3. Order Processing 4. Inventory Control 5. System Monitoring
Key Features	• Persistent cart functionality • Book search and filtering • Order history access • User profile management • Responsive mobile design	• Comprehensive data tables • CRUD operations • Real-time order status • User role management • Inventory tracking

D. USER INTERFACE & CONFIGURATION

2. FRONTEND ARCHITECTURE

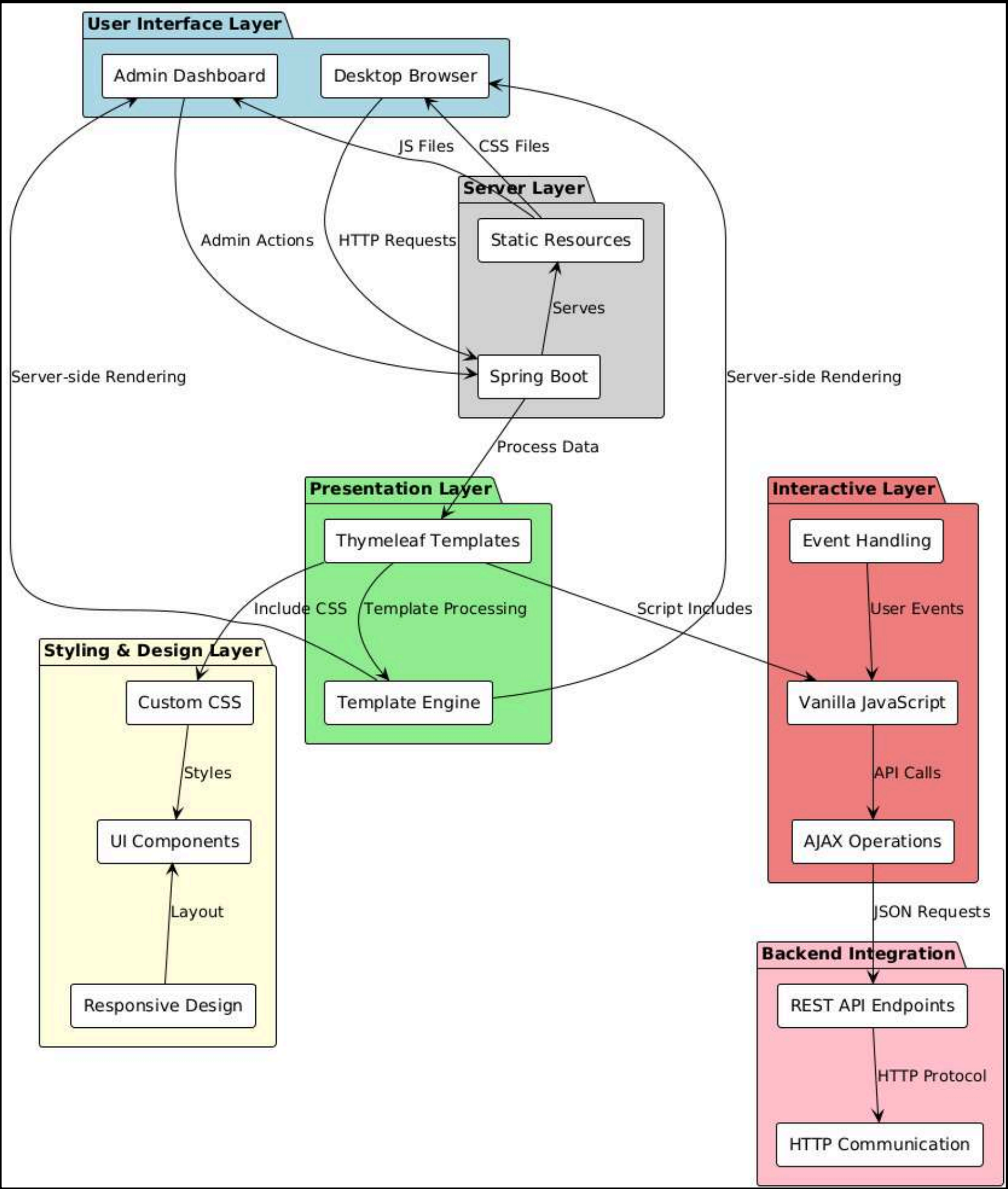
- SERVER-SIDE: THYMELEAF TEMPLATES + SPRING BOOT
- CLIENT-SIDE: VANILLA JAVASCRIPT + CUSTOM CSS
- COMMUNICATION: REST APIS + AJAX OPERATIONS
- DESIGN: RESPONSIVE UI WITH COMPONENT-BASED STYLING



D. USER INTERFACE & CONFIGURATION

3. FRONTEND WORKFLOW

Frontend Workflow Process			
Phase	Components	Action	Result
1. Initial Request	Desktop Browser → Spring Boot	HTTP Request	Route to Controller
2. Server Processing	Spring Boot → Template Engine	Data Fetch + Template Processing	Complete HTML Generation
3. Resource Serving	Static Resources → Client	CSS/JS File Delivery	Styled Interactive Page
4. Client Enhancement	JavaScript → Interactive Layer	Event Handling + AJAX	Dynamic User Experience
5. API Communication	REST Endpoints ↔ Backend	JSON Request/Response	Real-time Updates



D. USER INTERFACE & CONFIGURATION



4. CONFIGURATION & SETUP - OVERVIEW

- Applications to Install:**
- **Visual Studio Code** (Primary IDE)
 - **MySQL Server** (Database engine)
 - **MySQL Workbench** (Database management GUI)
 - **Git** (Version control)
 - **Java JDK 11+** (Runtime environment)
 - **Maven 3.6+** (Build tool - if not using wrapper)

Configuration Requirements			
Configuration Item	Required/ Recommended	PC-Specific Settings	Notes
Java JDK	JDK 11 or higher	Set JAVA_HOME environment variable	Check with <code>java -version</code>
Maven	3.6.0+	Set M2_HOME and add to PATH	Check with <code>mvn -version</code>
MySQL Server	8.0+	Database credentials (username/password)	Default port 3306
IDE	Visual Studio Code (recommended)	User preferences and workspace settings	Alternative: IntelliJ IDEA
Git	Latest version	Configure user.name and user.email	For version control

pom.xml Configuration			
Setting	Default Value	PC-Specific Changes	Location
Java Version	<code><java.version>11</java.version></code>	Match your installed JDK	<code><properties></code> section
Maven Version	Wrapper included	Update if using system Maven	Maven wrapper properties
MySQL Connector	<code>mysql-connector-j:8.2.0</code>	Update based on MySQL version	<code><dependencies></code>

application.properties Configuration			
Property	Default/Example	PC-Specific Changes	Purpose
Database URL	<code>spring.datasource.url=jdbc:mysql://localhost:3306/bookstore</code>	Change hostname/port if different	Database connection
Database Username	<code>spring.datasource.username=root</code>	Your MySQL username	Authentication
Database Password	<code>spring.datasource.password=yourpassword</code>	Your MySQL password	Authentication
Server Port	<code>server.port=8080</code>	Change if port is occupied	Application port
JPA Settings	<code>spring.jpa.hibernate.ddl-auto=update</code>	Keep as 'update' for development	Database schema

D. USER INTERFACE & CONFIGURATION

5. CONFIGURATION & SETUP - DETAILS

Phase 1: Environment Preparation

1. **Install Java JDK 11+, MySQL Server, Visual Studio Code**
2. **Configure Environment Variables**
Windows (System Properties → Environment Variables)
JAVA_HOME=C:\Program Files\Java\jdk-11.0.x
M2_HOME=C:\Program Files\Apache\maven-3.x.x *# If using system Maven*
PATH=%JAVA_HOME%\bin;%M2_HOME%\bin;%PATH%

Phase 2: Project Setup

3. **Clone/Download Project**
 - Your preferred directory to bookstoremanagementsystemversion2
4. **Configure Database Connection**
 - Open src/main/resources/application.properties
 - Update database credentials: username, password
5. **Verify pom.xml Settings**
 - Check Java version matches your installation
 - Ensure Maven wrapper is present (mvnw.cmd for Windows)

Phase 3: Database Initialization

6. **Create Database and Import Schema**
Navigate to project directory
cd ...\bookstoremanagementsystemversion2
Create database
mysql -u [username] -p -e "DROP DATABASE IF EXISTS bookstore; CREATE DATABASE bookstore;"
Import schema and data
mysql -u [username] -p bookstore < bookstore.sql
Verify import
mysql -u [username] -p bookstore -e "SELECT COUNT(*) as users FROM users; SELECT COUNT(*) as books FROM books;" *# If it turns out to be 7 users, 28 books, then it's correct*

Phase 4: Build and Run

7. **Clean and Install Dependencies**
.\mvnw.cmd clean install *# Using Maven wrapper*
8. **Start Application**
.\mvnw.cmd spring-boot:run *# Using Maven wrapper*
9. **Verify Application:** http://localhost:8080

E. USER EXPERIENCE DEMONSTRATION

